

FULL-STACK DEVELOPMENT INTERVIEW TASK

Work Order Management System

React + TypeScript | ASP.NET Core 8 | Entity Framework Core | SQL Server

1. Objective

Build a small production-oriented Work Order Management System that demonstrates frontend development, REST API design, database modelling, business rules, concurrency handling, performance awareness, and code quality.

2. Timebox & Deliverables

- Recommended implementation time: 2–3 hours.
- Submit the complete source code in a Git repository.
- Include a README with setup instructions, database configuration, assumptions, and design decisions.
- Provide API documentation through Swagger/OpenAPI or an equivalent mechanism.
- Include database migrations and/or scripts required to initialize the database.
- Include meaningful Git commits.

3. Technology Requirements

- Frontend: React + TypeScript
- UI: Material UI (MUI)
- Backend: ASP.NET Core 8 Web API
- ORM: Entity Framework Core
- Database: SQL Server
- API testing/documentation: Swagger/OpenAPI and/or Postman
- Version control: Git

4. Functional Requirements

Implement a Work Order Management module with the following capabilities:

- Create a work order.
- View work order details.
- Edit a work order.
- Assign a technician.
- Change work order status.
- View work order status history.
- Search, filter, sort, and paginate work orders.
- Enforce the defined work-order status transition rules.

5. Data Model

Customer

Required fields: Id, Name

Facility

Required fields: Id, CustomerId, Name, Location

Asset

Required fields: Id, FacilityId, AssetCode, Name, AssetType, Status

WorkOrder

Required fields: Id, AssetId, Title, Description, Priority, Status, AssignedTechnicianId, CreatedAt, UpdatedAt, RowVersion

WorkOrderHistory

Required fields: Id, WorkOrderId, OldStatus, NewStatus, ChangedBy, ChangedAt, Comments

Configure appropriate primary keys, foreign keys, relationships, constraints, and indexes using EF Core.

6. Work Order Status Workflow

Allowed transitions:

- New → Assigned
- Assigned → In Progress
- In Progress → Completed

Invalid transitions must be rejected by the backend. Every valid status change must create a WorkOrderHistory record.

7. REST API Requirements

Method	Endpoint	Purpose
POST	/api/workorders	Create work order
GET	/api/workorders	List work orders with pagination/filtering/sorting
GET	/api/workorders/{id}	Get work order details
PUT	/api/workorders/{id}	Update work order
DELETE	/api/workorders/{id}	Delete work order
PUT	/api/workorders/{id}/assign	Assign technician
PUT	/api/workorders/{id}/status	Change status
GET	/api/workorders/{id}/history	Get status history

The API should support:

- Server-side pagination.
- Search.
- Filtering by status, priority, facility, and technician.
- Sorting.
- DTOs rather than exposing EF Core entities directly.
- Input validation.
- Appropriate HTTP status codes.
- Consistent error responses.
- Global exception handling.
- Structured logging.
- Asynchronous database/API operations.

8. Concurrency Requirement

Two users may open the same work order at the same time. If User A updates the work order and User B subsequently attempts to save stale data, the system must prevent a silent lost update.

Implement optimistic concurrency. SQL Server rowversion with EF Core concurrency handling is one acceptable approach. Return a clear response when a concurrency conflict occurs.

9. React Application

Create a Work Orders screen containing:

- Search field.
- Status filter.
- Priority filter.
- Facility filter.
- Sortable work-order table.
- Server-side pagination.
- Create/Edit form.
- Assign Technician action.
- Change Status action.
- View History action.
- Loading, error, and empty states.

10. Performance Requirement

Assume the database contains up to 5 million work orders. The browser must not load all records at once.

- Use server-side pagination and filtering.
- Avoid unnecessarily large API responses.

- Explain how you would optimize React rendering for large datasets.
- Explain how you would identify whether a performance issue is caused by React, the API, the network, or SQL Server.

11. SQL Server Challenge

Assume the following query becomes slow as the data grows:

```
SELECT *
FROM WorkOrders
WHERE FacilityId = @FacilityId
AND Status = @Status
ORDER BY CreatedAt DESC;
```

Propose an appropriate index and explain why it would help. Explain how you would validate the improvement using the SQL Server execution plan and query metrics.

12. Architecture & Code Quality

- Use dependency injection appropriately.
- Keep controllers thin and place business logic in appropriate services/components.
- Separate API contracts (DTOs) from persistence entities.
- Use clear naming and maintainable project structure.
- Avoid duplicated business logic.
- Handle errors consistently.
- Use appropriate cancellation/async patterns.
- Include reasonable unit or integration tests where practical.

13. Optional Advanced Extensions

- Implement authentication and role/policy-based authorization.
- Restrict users to facilities they are authorized to access.
- Add real-time work-order status updates using SignalR.
- Add caching for suitable read-heavy endpoints.
- Add automated tests for status transitions and concurrency conflicts.
- Add Docker support.

14. Evaluation Areas

Area	Weight
React / TypeScript	20%
ASP.NET Core / API design	20%
EF Core / LINQ	15%
SQL Server / performance	15%
Business rules / concurrency	10%
Architecture / code quality	10%
Testing	5%
Documentation / Git practices	5%

15. Interview Review Questions

1. Why did you choose your particular React component and state structure?
2. Where is business validation performed, and why?
3. How does your concurrency mechanism prevent lost updates?
4. How would your solution behave with 5 million work orders?
5. Which indexes would you add and how would you verify them?
6. Where could N+1 queries occur in your implementation?
7. Why did you use Include versus projection in your EF Core queries?
8. How would you secure facility-level access if a user modifies facilityId manually?
9. What would you change before deploying this solution to production?

16. Submission Expectations

The solution should be runnable by another developer using the README instructions. The candidate should be prepared to explain architectural choices, trade-offs, performance decisions, and any known limitations during the technical interview.